



UNIVERSIDADE FEDERAL FLUMINENSE
CURSO DE CIÊNCIA DA COMPUTAÇÃO

CHRISTIANO GODINHO GUEDES PEREIRA
FÁBIO LUIZ GONÇALVES FILHO
THEODORO RIBEIRO DIAS

TRABALHO 1 - COMPILADORES
COMPILADORES

NITERÓI

2026

Material de referência

Abaixo, temos as regras da BNF da linguagem de programação C-Minus que foram utilizadas como referência para o trabalho atual. Além da BNF, também estão listadas as palavras reservadas da linguagem e a descrição por expressões regulares dos tokens da linguagem.

BNF Grammar for C-Minus

4. var-declaration $\rightarrow$ type-specifier ID ; | type-specifier ID [NUM] ;

18. expression $\rightarrow$ var = expression | simple-expression

23. addop $\rightarrow$ + | -

25. mulop $\rightarrow$ * | /

27. call $\rightarrow$ ID (args)

NUM = digit digit*

Special symbols: + - * / < <= > >= == != = ; , () [] { } /* */

ID = letter letter*

letter = a | .. | z | A | .. | Z

digit = 0 | .. | 9

Comments: /* ... */

Keywords: else, if, int, relop, return, void, while

Introdução

O trabalho desenvolvido consiste no primeiro estágio no desenvolvimento de um compilador simplificado para a linguagem de programação C-Minus, havendo anotações referentes a transformação de Expressões Regulares $\rightarrow$ Autômatos Finitos Não Determinísticos $\rightarrow$ Autômatos Finitos Determinísticos $\rightarrow$ Autômatos Finitos Determinísticos Mínimos, assim como o programa principal do compilador.

O programa está dividido em 3 e são eles: main, scan.c e scan.h. Suas funções são montar o corpo principal do programa e ler os programas passados para serem avaliados, separar os tokens e classificá-los e, por fim, define os tipos, algumas constantes e algumas variáveis que são usadas em outras partes do programa, respectivamente.

Para se compilar o programa usa-se o seguinte comando gcc main.c scan.c -o "Nome desejado", após isso basta executá-lo com .\"Nome do programa\" teste.cm passando um programa em C-Minus para que seja separado e classificado em tokens.

O processo de transformação de ER até AFD mínimo

Selecionadas as regras de produção e identificados os tokens a serem trabalhados, realizamos todo o processo de transformação de Expressões Regulares(ER) → Autômatos Finitos Não Determinísticos(AFND) → Autômatos Finitos Determinísticos(AFD) → Autômatos Finitos Determinísticos Mínimos(AFD Mínimo), seguindo o algoritmo de Thompson para a transformação de ER em AFND, o de ϵ -fecho para AFND para AFD, e o algoritmo de Hopcroft para a minimização do AFD. Os autômatos estão, com mais detalhes, disponíveis [neste painel do draw.io](#).

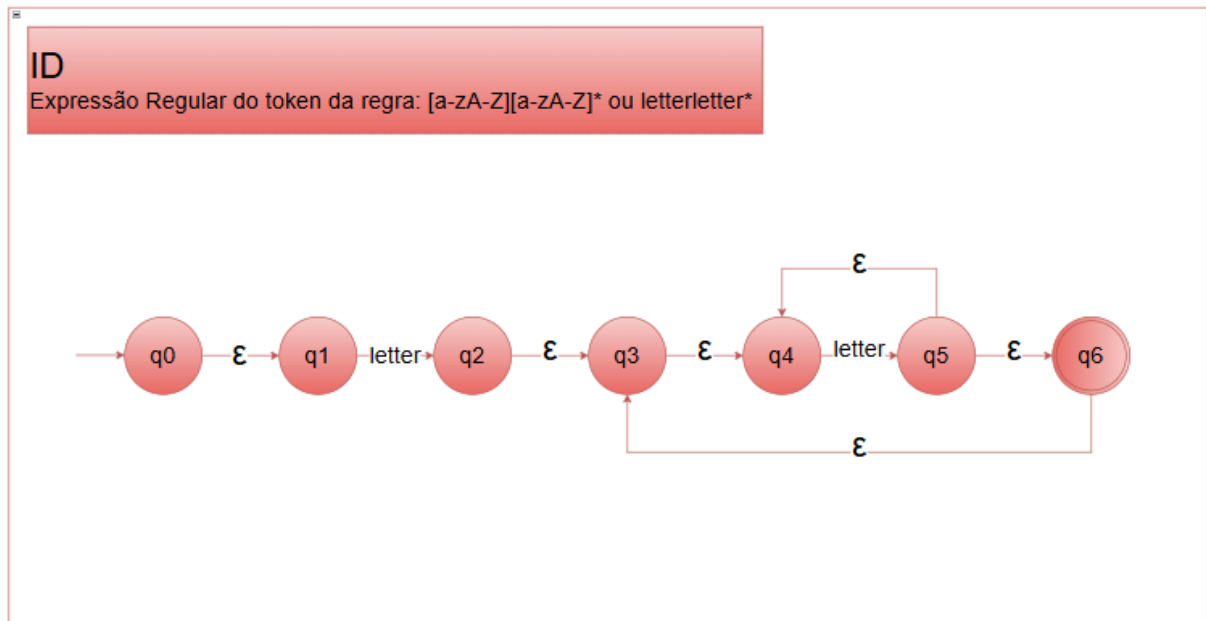


Imagem 1: ER para AFND pelo algoritmo de Thompson

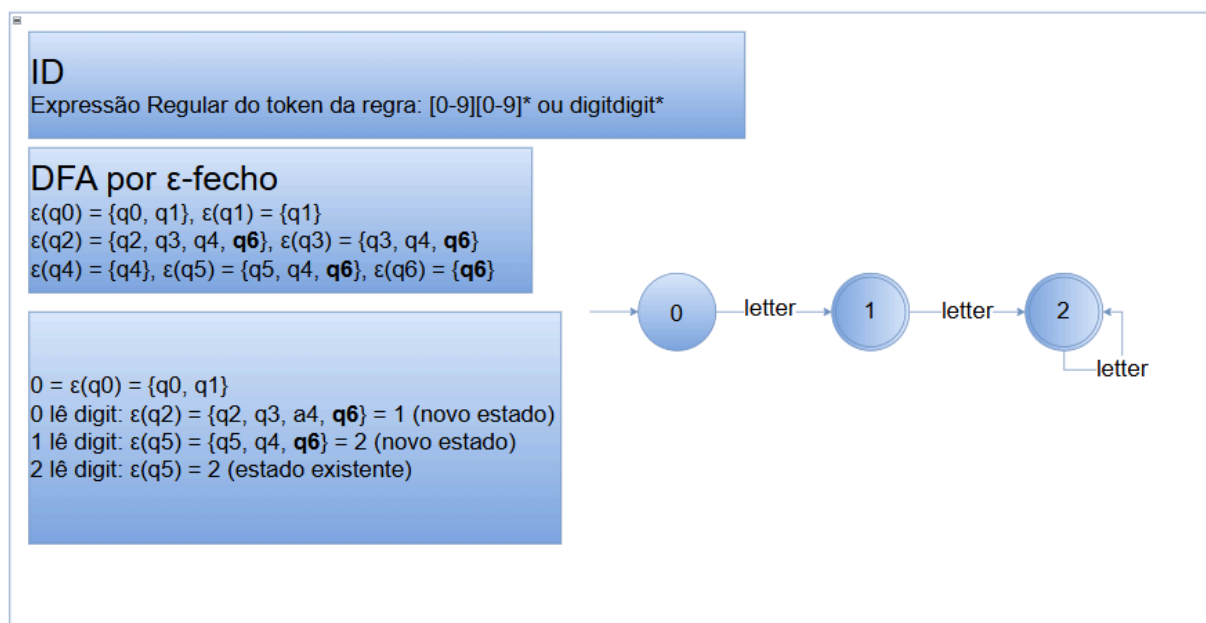


Imagem 2: AFND para AFD pelo algoritmo de ϵ -fecho

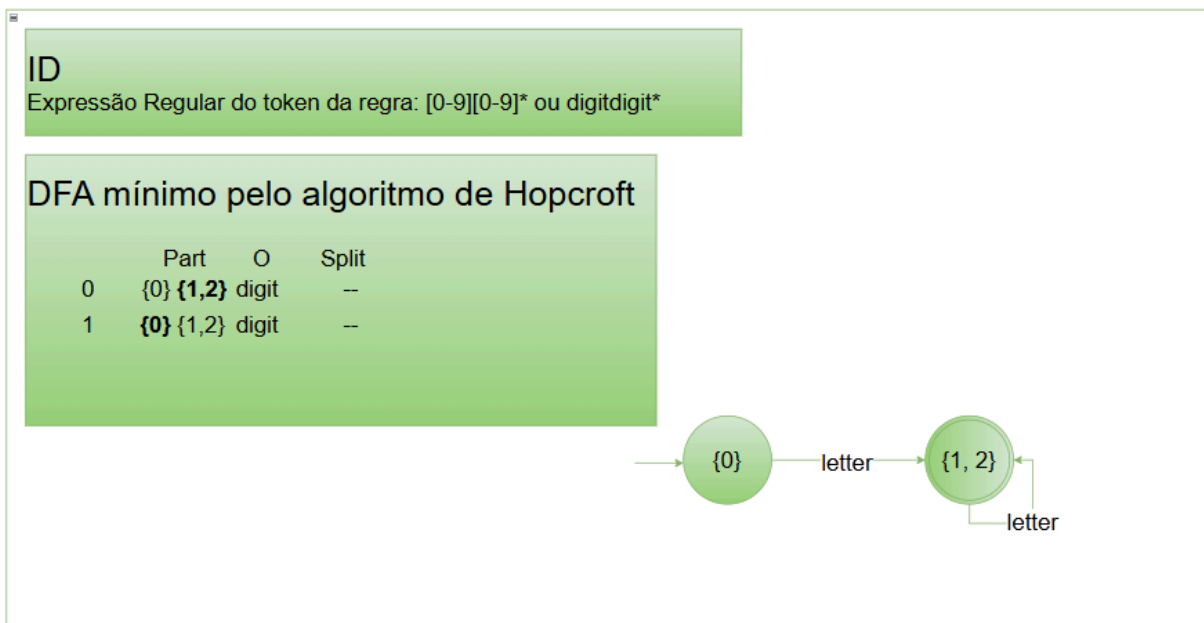


Imagem 3: AFND para AFD pelo algoritmo de Hopcroft

Após a construção de todos os respectivos AFDs Mínimos, realizamos a concatenação em um único autômato, que chamamos de AFD unificado mínimo. Com este autômato montado, iniciamos o desenvolvimento do scanner.

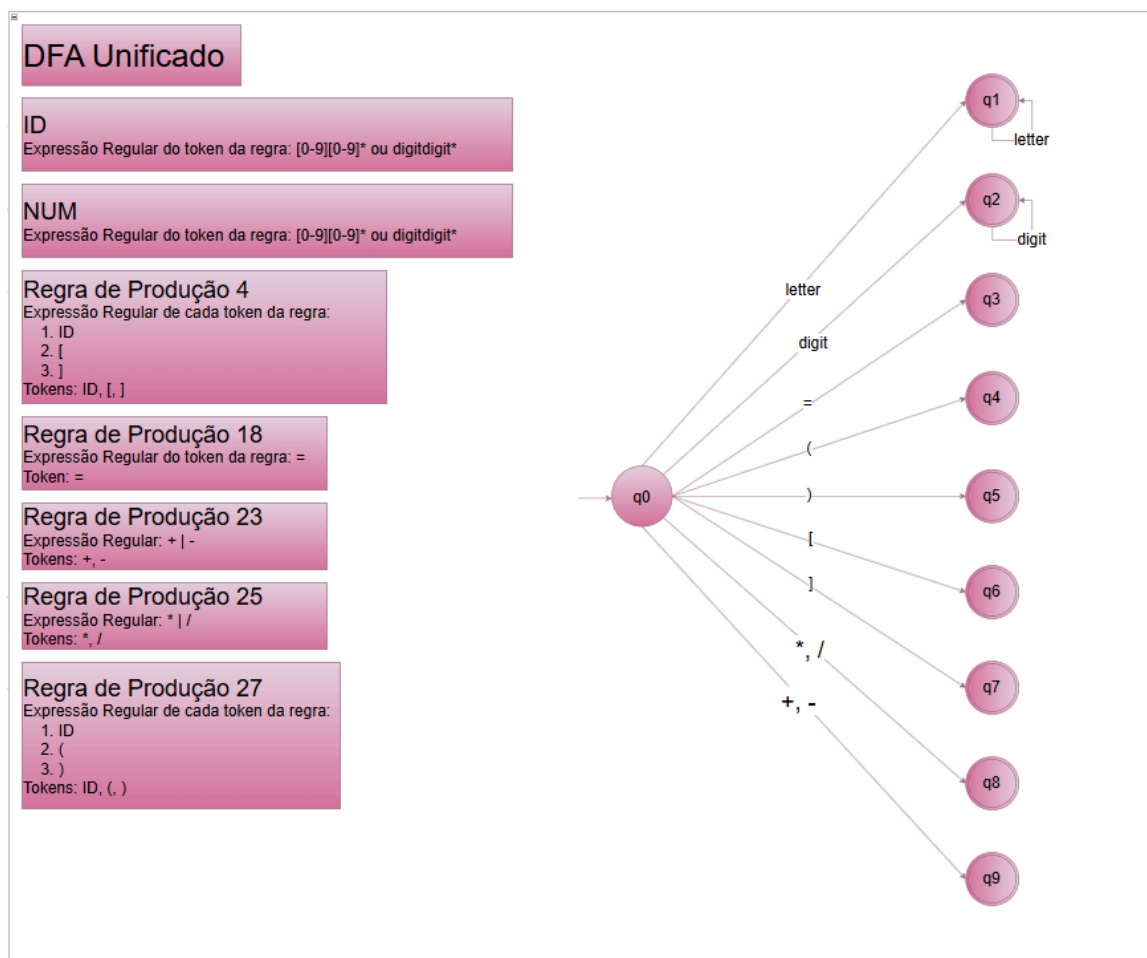


Imagem 4: AFD unificado mínimo

Como as regras seleccionadas pelo grupo não abrangem todos os tokens da linguagem, o AFD unificado mínimo não possui todos os tokens possíveis, como (,), <, >, etc, mas todos os tokens se encontram implementados no scanner. Assim, como leitura de comentários.

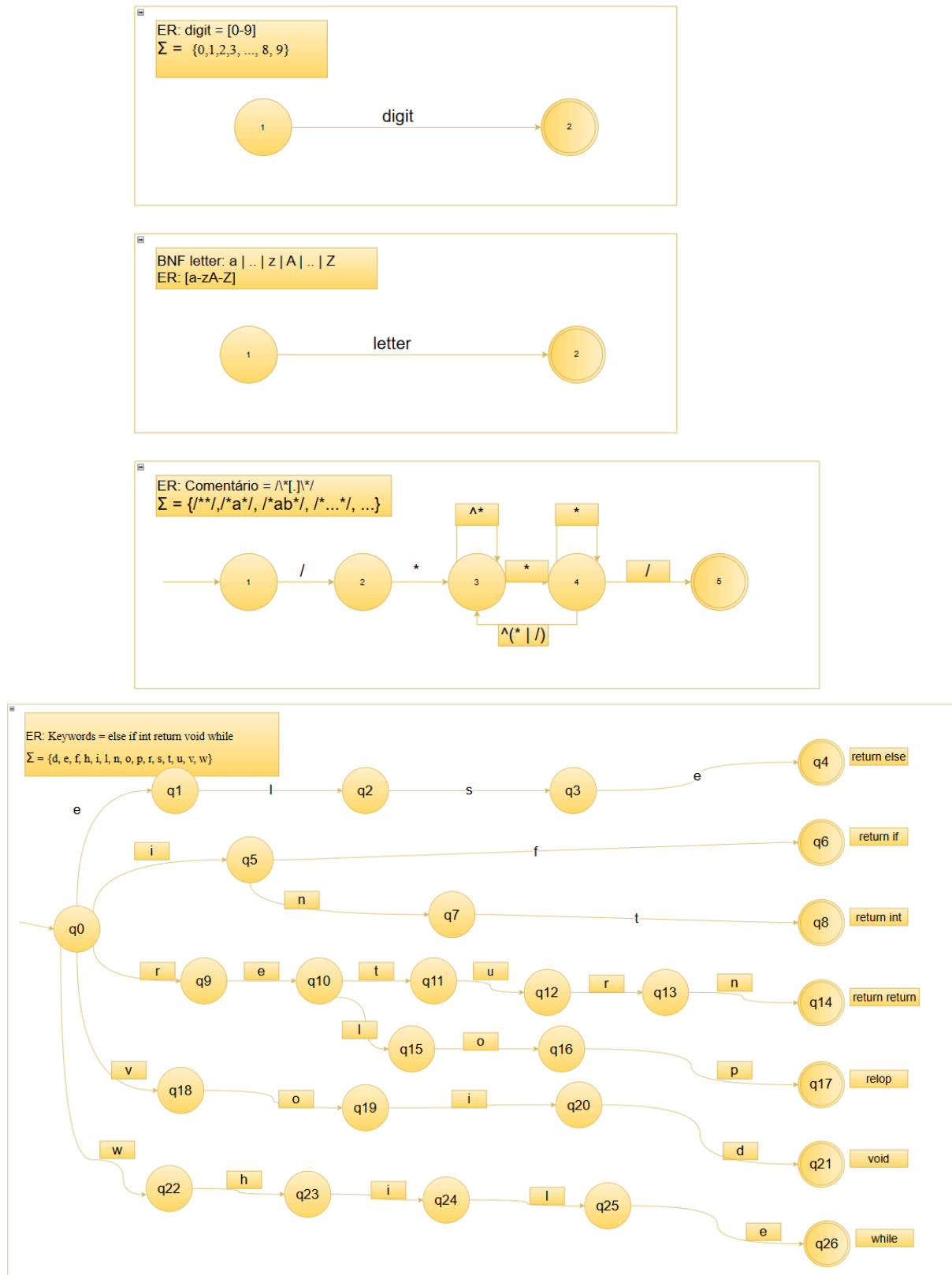


Imagem 5: AFDs auxiliares

O Programa

Foram selecionados trechos considerados mais importantes do código para serem ressaltados, o trecho abaixo define os tipos que serão aceitos e classifica os tokens dos programas em C-Minus.

```
C/C++
typedef enum
{
    ENDFILE,
    ERROR,

    ELSE,
    IF,
    INT,
    RETURN,
    VOID,
    WHILE,

    IDENTIFICADOR,
    NUMERO,

    ATRIBUICAO,
    IGUAL,
    MENOR,
    MENOR_IGUAL,
    MAIOR,
    MAIOR_IGUAL,
    DIFERENTE,

    MAIS,
    MENOS,
    VEZES,
    DIVISAO,

    PONTO_VIRGULA,
    VIRGULA,
    ABRE_PARENTESE,
    FECHA_PARENTESE,
    ABRE_COLCHETE,
    FECHA_COLCHETE,
    ABRE_CHAVE,
    FECHA_CHAVE
} TokenType;
```

Abaixo, vemos a lógica que trata de abrir e ler os arquivos fonte, trata o caso de estar vazio, e imprime os dados classificados.

C/C++

```
int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Uso: %s <arquivo.cm>\n", argv[0]);
        return 1;
    }

    Fonte = fopen(argv[1], "r");
    if (Fonte == NULL) {
        perror("Arquivo vazio?");
        return 1;
    }

    listing = stdout;
    TokenType token;

    printf("Linha\tToken\t\tLexema\n");

    do {
        token = getToken();
        if (token != ENDFILE) {
            printf("%d\t%-12s\t%s\n", lineno, tokenNames[token], tokenString);
        }
    } while (token != ENDFILE);

    fclose(Fonte);
    return 0;
}
```

O último trecho mostra como o programa consegue saber qual é o token lido, fazendo uma comparação letra a letra com a tabela ASCII e testando com os valores esperados para identificá-lo.

C/C++

```
# [...]

case '>':

    p = pegaChar();

    if (p == '=')
    {

        tokenString[1] = '=';

        tokenString[2] = '\0';
```

```

        return MAIOR_IGUAL;

    }

    devolveChar(p);

    return MAIOR;

case '!':

    p = pegaChar();

    if (p == '=')

    {

        tokenString[1] = '=';

        tokenString[2] = '\\0';

        return DIFERENTE;

    }

# [...]

```

O arquivo Flex

O arquivo feito para ser lido pelo flex foi feito seguindo a mesma base da implementação acima, e pode ser localizado na pasta **scanner-flex**, com o nome de **scanner.l**.

Houveram dificuldades em lidar com a sintaxe da ferramenta, mas a partir do achado de uma referência nos códigos do livro do louden, e no site oficial desta, os obstáculos foram superados.

O arquivo então é compilado e executado normalmente.

Arquivos de Teste

As implementações foram testadas com dois arquivos de teste, sem e com erros, de nomes **teste_sem_erros.cm** e **teste_com_erros.cm**. Abaixo temos as implementações sem erros e com erros, respectivamente.

```

C/C++
/*
  Arquivo de teste para Analisador Léxico da linguagem C-
  Contém funções, estruturas de controle, operadores e tipos básicos.
*/

```

```
int globalvar;

/* Função para calcular o fatorial de um número */
int fatorial(int n) {
    int resultado;

    if (n <= 1) {
        resultado = 1;
    } else {
        resultado = n * fatorial(n - 1);
    }

    return resultado;
}

/* Função principal */
void main(void) {
    int x;
    int y;
    int vetor[10];

    x = 5;
    y = fatorial(x);

    /* Testando operadores relacionais e lógicos/condicionais */
    if (y >= 120) {
        vetor[0] = 1;
    } else {
        vetor[0] = 0;
    }

    /* Testando laço de repetição */
    while (x > 0) {
        x = x - 1;
    }

    output(y);
}
```

```

C/C++
/*
Arquivo de teste COM ERROS LÉXICOS para a linguagem C-
Este arquivo foi criado para verificar se o seu analisador léxico
consegue identificar e reportar corretamente falhas de símbolos.
*/

int main(void) {
int 123var;      /* Erro 1: Identificador inválido começando com número */
int x;

x = 10;

/* Erro 2: Símbolo/caractere desconhecido ou inválido na linguagem C- */
x = x @ 2;

/* Erro 3: Operador inválido ou incompleto (ex: & sozinho se não suportado) */
if (x # 5) {
    x = x $ 1;
}

/* Erro 4: Comentário de bloco não fechado (Lexer deve acusar EOF no
comentário) */
/* Este comentário nunca termina...

return 0;

}

```

Processo de Teste

Os testes foram feitos **diretamente no terminal**, passando o nome do arquivo como parâmetros de linha de comando. Resultados de execução foram coletados e estão dispostos a seguir.

Resultados implementação manual

```
$ ./teste_scan teste_sem_erros.cm
Linha Token Lexema
6 INT int
6 IDENTIFICADOR globalvar
6 PONTO_VIRGULA ;
9 INT int
9 IDENTIFICADOR fatorial
9 ABRE_PARENTESE (
9 INT int
9 IDENTIFICADOR n
9 FECHA_PARENTESE )
9 ABRE_CHAVE {
10 INT int
10 IDENTIFICADOR resultado
10 PONTO_VIRGULA ;
12 IF if
12 ABRE_PARENTESE (
12 IDENTIFICADOR n
12 MENOR_IGUAL <=
12 NUMERO 1
12 FECHA_PARENTESE )
12 ABRE_CHAVE {
13 IDENTIFICADOR resultado
13 ATRIBUICAO =
13 NUMERO 1
13 PONTO_VIRGULA ;
14 FECHA_CHAVE }
14 ELSE else
14 ABRE_CHAVE {
15 IDENTIFICADOR resultado
15 ATRIBUICAO =
15 IDENTIFICADOR n
15 VEZES *
15 IDENTIFICADOR fatorial
15 ABRE_PARENTESE (
15 IDENTIFICADOR n
15 MENOS -
15 NUMERO 1
15 FECHA_PARENTESE )
15 PONTO_VIRGULA ;
16 FECHA_CHAVE }
18 RETURN return
18 IDENTIFICADOR resultado
18 PONTO_VIRGULA ;
19 FECHA_CHAVE }
19 VOID void

19 FECHA_CHAVE }
22 VOID void
22 IDENTIFICADOR main
22 ABRE_PARENTESE (
22 VOID void
22 FECHA_PARENTESE )
22 ABRE_CHAVE {
23 INT int
23 IDENTIFICADOR x
23 PONTO_VIRGULA ;
24 INT int
24 IDENTIFICADOR y
24 PONTO_VIRGULA ;
25 INT int
25 IDENTIFICADOR vetor
25 ABRE_COLCHETE [
25 NUMERO 10
25 FECHA_COLCHETE ]
25 PONTO_VIRGULA ;
27 IDENTIFICADOR x
27 ATRIBUICAO =
27 NUMERO 5
27 PONTO_VIRGULA ;
28 IDENTIFICADOR y
28 ATRIBUICAO =
28 IDENTIFICADOR fatorial
28 ABRE_PARENTESE (
28 IDENTIFICADOR x
28 FECHA_PARENTESE )
28 PONTO_VIRGULA ;
31 IF if
31 ABRE_PARENTESE (
31 IDENTIFICADOR y
31 MAIOR_IGUAL >=
31 NUMERO 120
31 FECHA_PARENTESE )
31 ABRE_CHAVE {
32 IDENTIFICADOR vetor
32 ABRE_COLCHETE [
32 NUMERO 0
32 FECHA_COLCHETE ]
32 ATRIBUICAO =
32 NUMERO 1
32 PONTO_VIRGULA ;
33 FECHA_CHAVE }
33 ELSE else

31 IF if
31 ABRE_PARENTESE (
31 IDENTIFICADOR y
31 MAIOR_IGUAL >=
31 NUMERO 120
31 FECHA_PARENTESE )
31 ABRE_CHAVE {
32 IDENTIFICADOR vetor
32 ABRE_COLCHETE [
32 NUMERO 0
32 FECHA_COLCHETE ]
32 ATRIBUICAO =
32 NUMERO 1
32 PONTO_VIRGULA ;
35 FECHA_CHAVE }
38 WHILE while
38 ABRE_PARENTESE (
38 IDENTIFICADOR x
38 MAIOR >
38 NUMERO 0
38 FECHA_PARENTESE )
38 ABRE_CHAVE {
39 IDENTIFICADOR x
39 ATRIBUICAO =
39 IDENTIFICADOR x
39 MENOS -
39 NUMERO 1
39 PONTO_VIRGULA ;
40 FECHA_CHAVE }
42 IDENTIFICADOR output
42 ABRE_PARENTESE (
42 IDENTIFICADOR y
42 FECHA_PARENTESE )
42 PONTO_VIRGULA ;
43 FECHA_CHAVE }

$ ./teste_scan teste_com_erros.cm
Linha Token Lexema
7 INT int
7 IDENTIFICADOR main
7 ABRE_PARENTESE (
7 VOID void
7 FECHA_PARENTESE )
7 ABRE_CHAVE {
8 INT int
8 NUMERO 123
8 IDENTIFICADOR var
8 PONTO_VIRGULA ;
9 INT int
9 IDENTIFICADOR x
9 PONTO_VIRGULA ;
11 IDENTIFICADOR x
11 ATRIBUICAO =
11 NUMERO 10
11 PONTO_VIRGULA ;
14 IDENTIFICADOR x
14 ATRIBUICAO =
14 IDENTIFICADOR x
14 ERROR @
14 NUMERO 2
14 PONTO_VIRGULA ;
17 IF if
17 ABRE_PARENTESE (
17 IDENTIFICADOR x
17 ERROR #
17 NUMERO 5
17 FECHA_PARENTESE )
17 ABRE_CHAVE {
18 IDENTIFICADOR x
18 ATRIBUICAO =
18 IDENTIFICADOR x
18 ERROR $
18 NUMERO 1
18 PONTO_VIRGULA ;
19 FECHA_CHAVE }

$
```

Resultados implementação com Flex (com e sem erros)

```
$ ./scanner-flex/flex_exec teste_sem_erros.cm
3: reserved word: int
3: ID, name= globalvar
3: SEMI ;
6: reserved word: int
6: ID, name= fatorial
6: (
6: reserved word: int
6: ID, name= n
6: )
6: {
7: reserved word: int
7: ID, name= resultado
7: SEMI ;
9: reserved word: if
9: (
9: ID, name= n
9: <=
9: NUM, val= 1
9: )
9: {
10: ID, name= resultado
10: =
10: NUM, val= 1
10: SEMI ;
11: }
11: reserved word: else
11: {
12: ID, name= resultado
12: =
12: ID, name= n
12: *
12: ID, name= fatorial
12: (
12: ID, name= n
12: -
12: NUM, val= 1
12: )
12: SEMI ;
13: }
15: reserved word: return
15: ID, name= resultado
15: SEMI ;
16: }
19: reserved word: void
19: ID, name= main
```

```
19: reserved word: void
19: ID, name= main
19: (
19: reserved word: void
19: )
19: {
20: reserved word: int
20: ID, name= x
20: SEMI ;
21: reserved word: int
21: ID, name= y
21: SEMI ;
22: reserved word: int
22: ID, name= vetor
22: [
22: NUM, val= 10
22: ]
22: SEMI ;
24: ID, name= x
24: =
24: NUM, val= 5
24: SEMI ;
25: ID, name= y
25: =
25: ID, name= fatorial
25: (
25: ID, name= x
25: )
25: SEMI ;
28: reserved word: if
28: (
28: ID, name= y
28: >=
28: NUM, val= 120
28: )
28: {
29: ID, name= vetor
29: [
29: NUM, val= 0
29: ]
29: =
29: NUM, val= 1
29: SEMI ;
30: }
30: reserved word: else
30: {
```

```
28: reserved word: if
28: (
28: ID, name= y
28: >=
28: NUM, val= 120
28: )
28: {
29: ID, name= vetor
29: [
29: NUM, val= 0
29: ]
29: =
29: NUM, val= 1
29: SEMI ;
30: }
30: reserved word: else
30: {
31: ID, name= vetor
31: [
31: NUM, val= 0
31: ]
31: =
31: NUM, val= 0
31: SEMI ;
32: }
35: reserved word: while
35: (
35: ID, name= x
35: >
35: NUM, val= 0
35: )
35: {
36: ID, name= x
36: =
36: ID, name= x
36: -
36: NUM, val= 1
36: SEMI ;
37: }
39: ID, name= output
39: (
39: ID, name= y
39: )
39: SEMI ;
40: }
```

\$ □

```
$ ./flex_exec ../teste_com_erros.cm
3: reserved word: int
3: ID, name= main
3: (
3: reserved word: void
3: )
3: {
4: reserved word: int
4: LEXICAL ERROR: 123var
4: SEMI ;
5: reserved word: int
5: ID, name= x
5: SEMI ;
7: ID, name= x
7: =
7: NUM, val= 10
7: SEMI ;
10: ID, name= x
10: =
10: ID, name= x
10: LEXICAL ERROR: @
10: NUM, val= 2
10: SEMI ;
13: reserved word: if
13: (
13: ID, name= x
13: LEXICAL ERROR: #
13: NUM, val= 5
13: )
13: {
14: ID, name= x
14: =
14: ID, name= x
14: LEXICAL ERROR: $
14: NUM, val= 1
14: SEMI ;
15: }
```

Erro léxico: Comentário não fechado no fim do arquivo

\$ □

Prompts de IA usados

Foram usados prompts para:

- Entender a diferença entre os tokens ENDFILE e ERROR;
- Reescrever o arquivo scanner.l com nomes mais descritivos pros tokens, criando o globals.h e util.c com base no livro do louden.

Link do Repositório GitHub

O repositório do trabalho está disponível [neste link](https://github.com/lugon-hex/trabalho-compiladores/)
(<https://github.com/lugon-hex/trabalho-compiladores/>)

Link do painel do Draw.io

O repositório do trabalho está disponível [neste link](https://drive.google.com/file/d/1sL1U-NBpsE-qKwtRaLLmTDQXIgckhA0V/view?usp=sharing)
<https://drive.google.com/file/d/1sL1U-NBpsE-qKwtRaLLmTDQXIgckhA0V/view?usp=sharing>